

1. Import libraries

```
In [6]: import pandas as pd
import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt
```

2. Load Titanic data

```
In [2]: kashti = sns.load_dataset('titanic')
```

```
In [7]: kashti.head()
```

```
Out[7]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_m
0	0	3	male	22.0	1	0	7.2500	S	Third	man	T
1	1	1	female	38.0	1	0	71.2833	C	First	woman	Fa
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	Fa
3	1	1	female	35.0	1	0	53.1000	S	First	woman	Fa
4	0	3	male	35.0	0	0	8.0500	S	Third	man	T

```
In [8]: kashti.tail()
```

```
Out[8]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult
886	0	2	male	27.0	0	0	13.00	S	Second	man	
887	1	1	female	19.0	0	0	30.00	S	First	woman	
888	0	3	female	NaN	1	2	23.45	S	Third	woman	
889	1	1	male	26.0	0	0	30.00	C	First	man	
890	0	3	male	32.0	0	0	7.75	Q	Third	man	

```
In [9]: kashti.shape
```

```
Out[9]: (891, 15)
```

```
In [10]: kashti.columns
```

```
Out[10]: Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',
               'embarked', 'class', 'who', 'adult_male', 'deck', 'embark_town',
               'alive', 'alone'],
              dtype='object')
```

```
In [11]: kashti.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column          Non-Null Count  Dtype  
---  -
0   survived        891 non-null   int64  
1   pclass          891 non-null   int64  
2   sex             891 non-null   object  
3   age             714 non-null   float64 
4   sibsp          891 non-null   int64  
5   parch          891 non-null   int64  
6   fare           891 non-null   float64 
7   embarked       889 non-null   object  
8   class          891 non-null   category
9   who            891 non-null   object  
10  adult_male     891 non-null   bool    
11  deck          203 non-null   category
12  embark_town    889 non-null   object  
13  alive         891 non-null   object  
14  alone         891 non-null   bool    
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB
```

```
In [3]: kashti[kashti["age"] < 5].head()
```

```
Out[3]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_n
7	0	3	male	2.0	3	1	21.0750	S	Third	child	F
10	1	3	female	4.0	1	1	16.7000	S	Third	child	F
16	0	3	male	2.0	4	1	29.1250	Q	Third	child	F
43	1	2	female	3.0	1	2	41.5792	C	Second	child	F
63	0	3	male	4.0	3	2	27.9000	S	Third	child	F

```
In [4]: kashti[kashti["age"] < 1] # anyone under age 1 year traveled free?
```

```
Out[4]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_n
78	1	2	male	0.83	0	2	29.0000	S	Second	child	
305	1	1	male	0.92	1	2	151.5500	S	First	child	
469	1	3	female	0.75	2	1	19.2583	C	Third	child	
644	1	3	female	0.75	2	1	19.2583	C	Third	child	
755	1	2	male	0.67	1	1	14.5000	S	Second	child	
803	1	3	male	0.42	0	1	8.5167	C	Third	child	
831	1	2	male	0.83	1	1	18.7500	S	Second	child	

```
In [5]: kashti[kashti["fare"] < 5].head()
```

```
Out[5]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male
179	0	3	male	36.0	0	0	0.0	S	Third	man	True
263	0	1	male	40.0	0	0	0.0	S	First	man	True
271	1	3	male	25.0	0	0	0.0	S	Third	man	True
277	0	2	male	NaN	0	0	0.0	S	Second	man	True
302	0	3	male	19.0	0	0	0.0	S	Third	man	True

3. Basic understanding

```
In [12]: kashti.describe()
```

```
Out[12]:
```

	survived	pclass	age	sibsp	parch	fare
count	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

```
In [13]: kashti.describe(include="object")
```

```
Out[13]:
```

	sex	embarked	who	embark_town	alive
count	891	889	891	889	891
unique	2	3	3	3	2
top	male	S	man	Southampton	no
freq	577	644	537	644	549

```
In [16]: kashti.dtypes
```

```
Out[16]: survived      int64
         pclass        int64
         sex           object
         age           float64
         sibsp         int64
         parch         int64
         fare          float64
         embarked      object
         class         category
         who           object
         adult_male     bool
         deck          category
         embark_town    object
         alive          object
         alone          bool
         dtype: object
```

```
In [18]: kashti.nunique()
```

```
Out[18]: survived      2
         pclass        3
         sex           2
         age           88
         sibsp         7
         parch         7
         fare          248
         embarked      3
         class         3
         who           3
         adult_male     2
         deck          7
         embark_town    3
         alive          2
         alone          2
         dtype: int64
```

4. Missing values

```
In [19]: kashti.isnull().sum()
```

```
Out[19]: survived      0
         pclass        0
         sex           0
         age          177
         sibsp         0
         parch         0
         fare          0
         embarked      2
         class         0
         who           0
         adult_male    0
         deck          688
         embark_town    2
         alive         0
         alone         0
         dtype: int64
```

```
In [20]: missing = kashti.isnull().sum().sort_values(ascending=False)

missing_percent = (kashti.isnull().sum() / len(kashti) * 100).sort_values(
    ascending=False
)

missing_Kashti = pd.DataFrame({
    "Missing": missing,
    "Percentage": missing_percent
})

missing_Kashti
```

Out[20]:

	Missing	Percentage
deck	688	77.216611
age	177	19.865320
embarked	2	0.224467
embark_town	2	0.224467
sex	0	0.000000
pclass	0	0.000000
survived	0	0.000000
fare	0	0.000000
parch	0	0.000000
sibsp	0	0.000000
class	0	0.000000
adult_male	0	0.000000
who	0	0.000000
alive	0	0.000000
alone	0	0.000000

5. Duplicate records

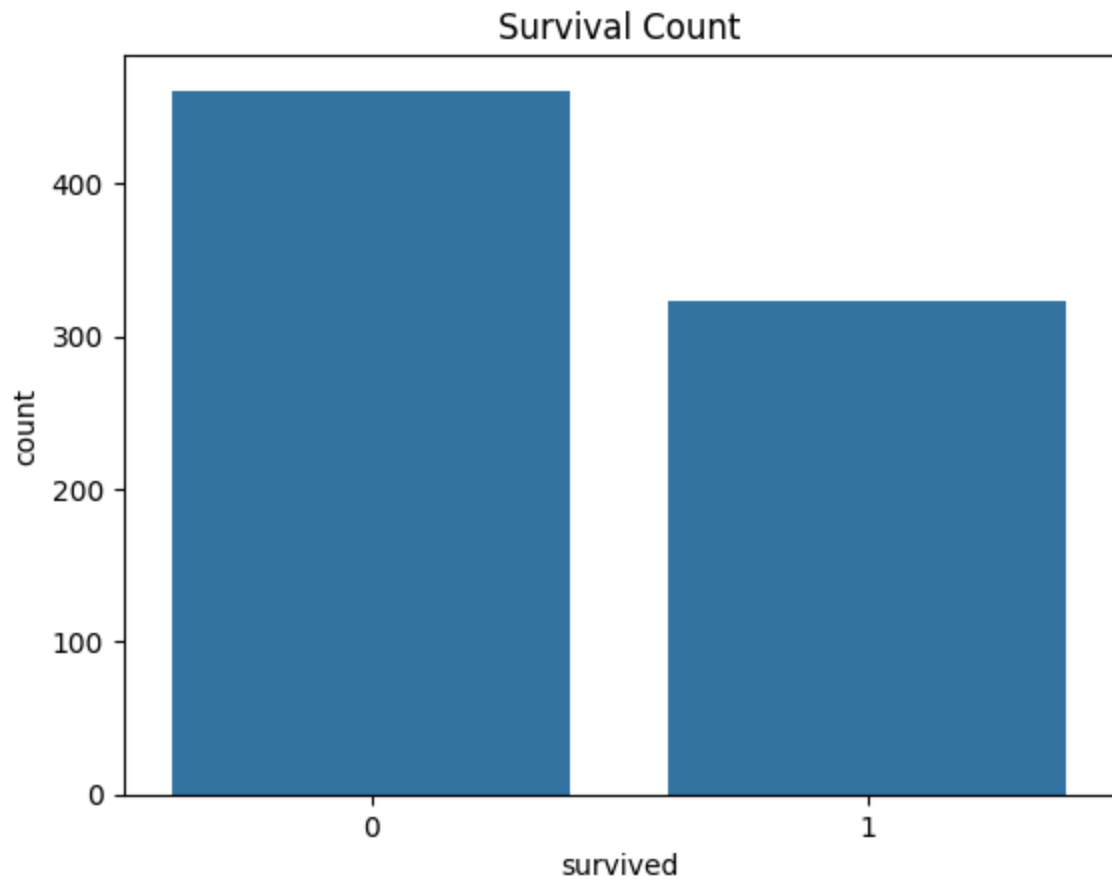
```
In [21]: kashti.duplicated().sum()
```

Out[21]: np.int64(107)

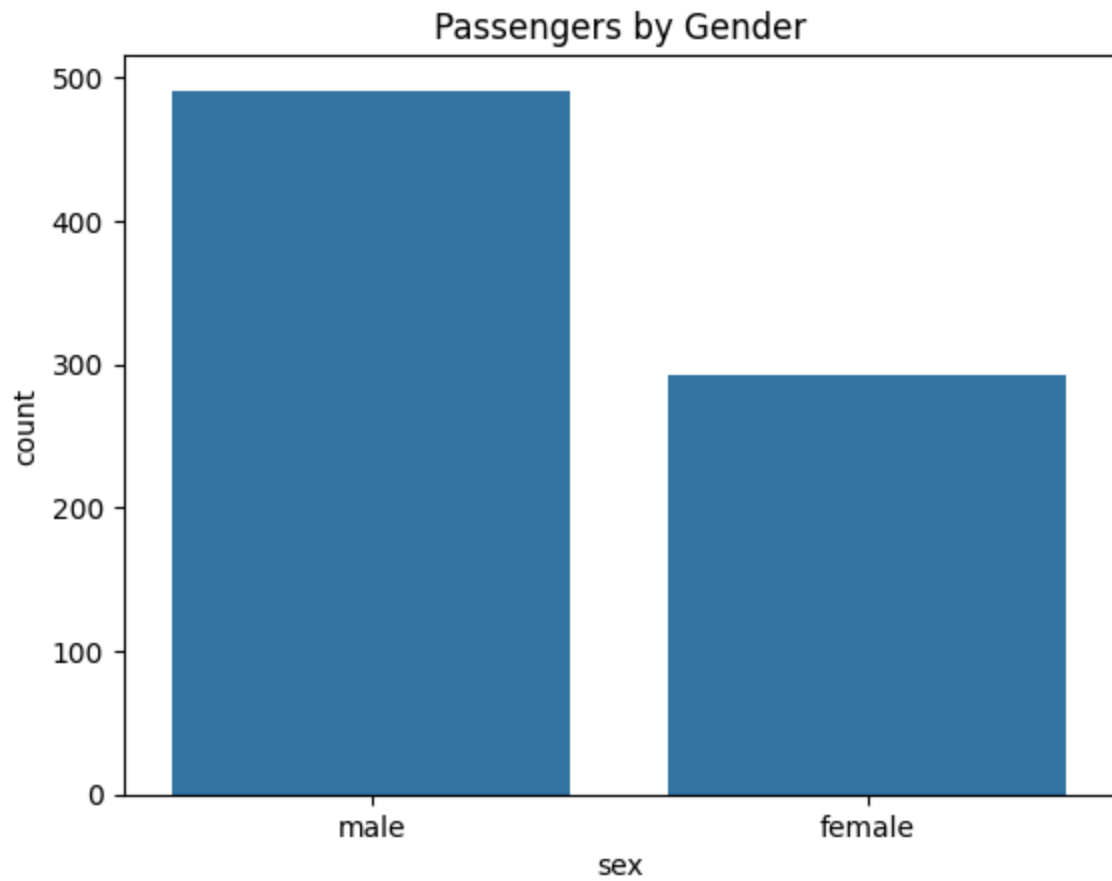
```
In [22]: kashti = kashti.drop_duplicates()
```

6. Univariate Analysis

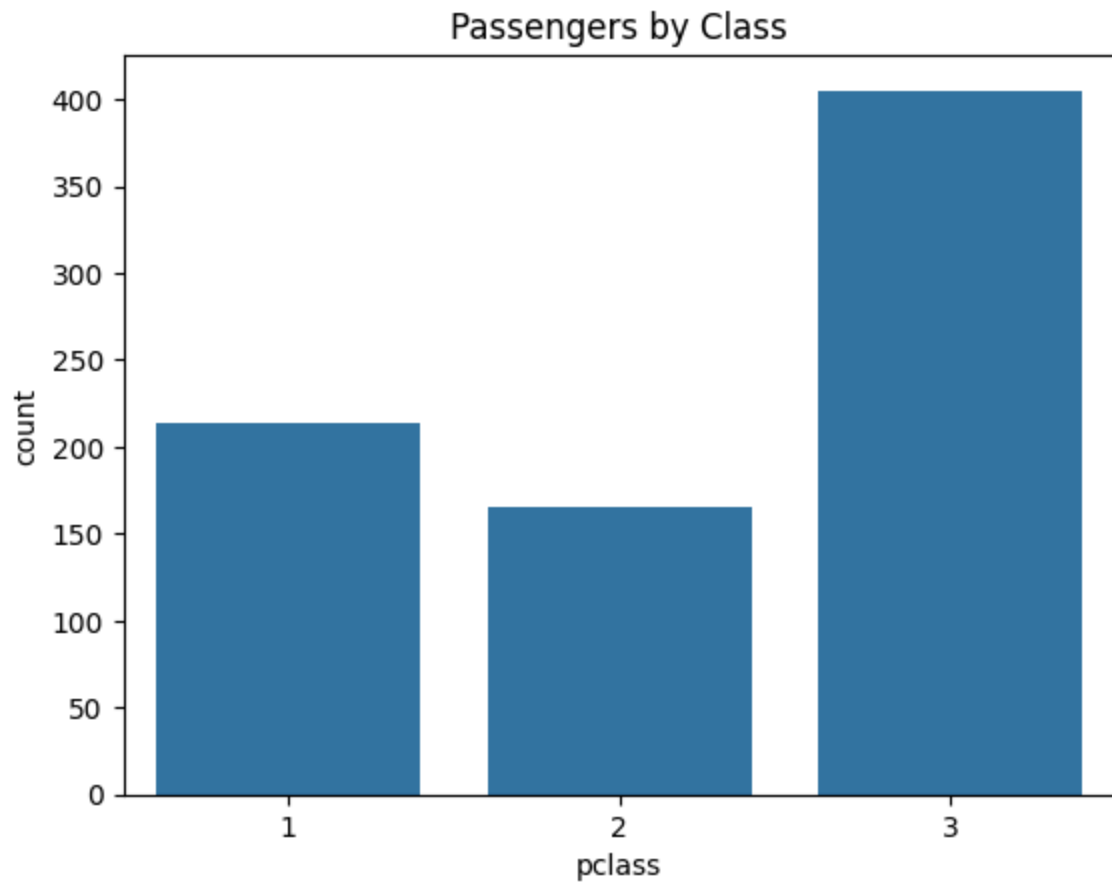
```
In [24]: #Survival
sns.countplot(data=kashti, x="survived")
plt.title("Survival Count")
plt.show()
```



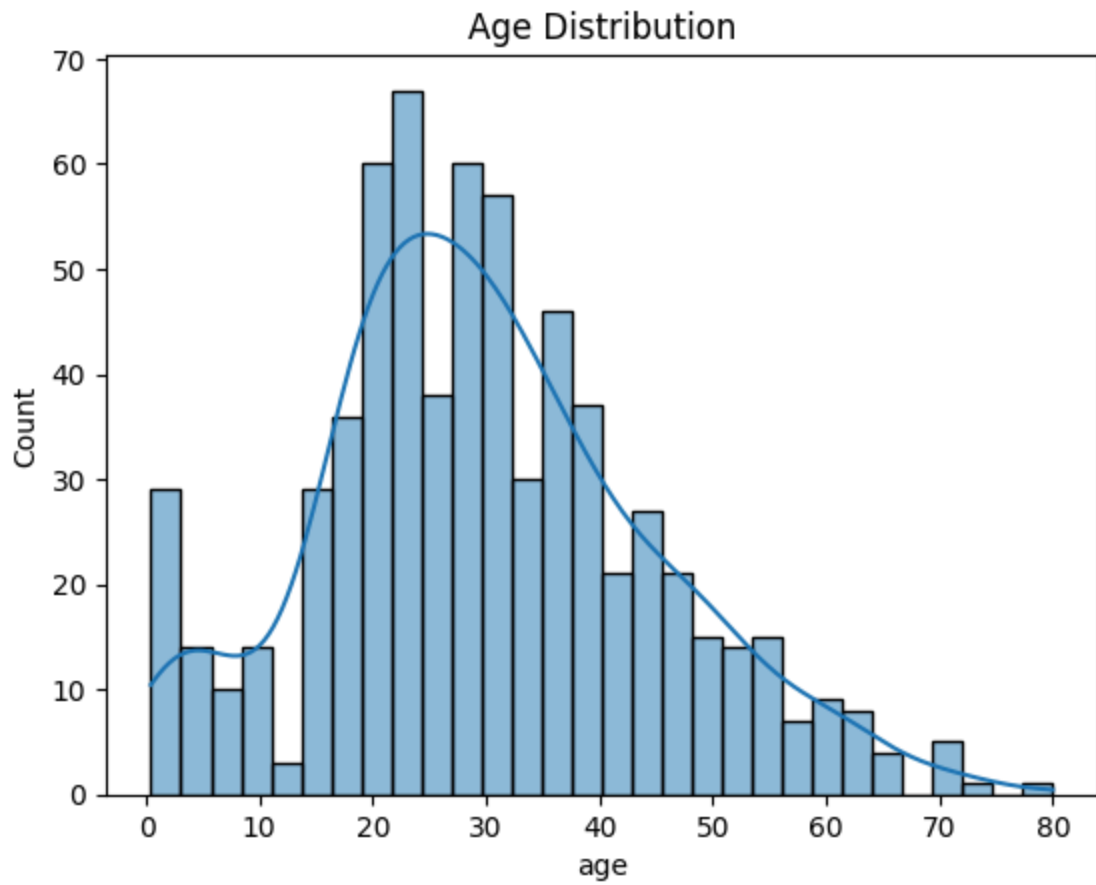
```
In [27]: #Gender
sns.countplot(data=kashti, x="sex")
plt.title("Passengers by Gender")
plt.show()
```



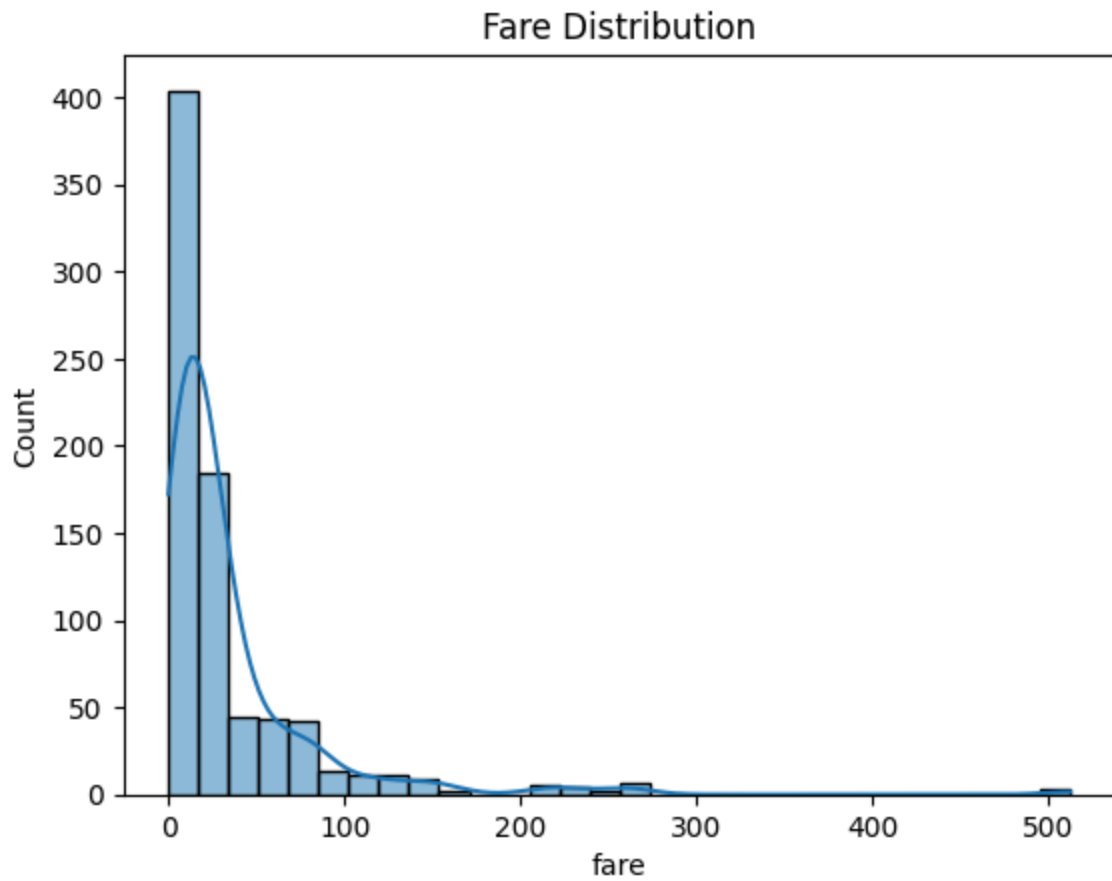
```
In [29]: #Passenger class
sns.countplot(data=kashti, x="pclass")
plt.title("Passengers by Class")
plt.show()
```

```
In [30]: #Age distribution
sns.histplot(data=kashti, x="age", bins=30, kde=True)
plt.title("Age Distribution")
plt.show()
```

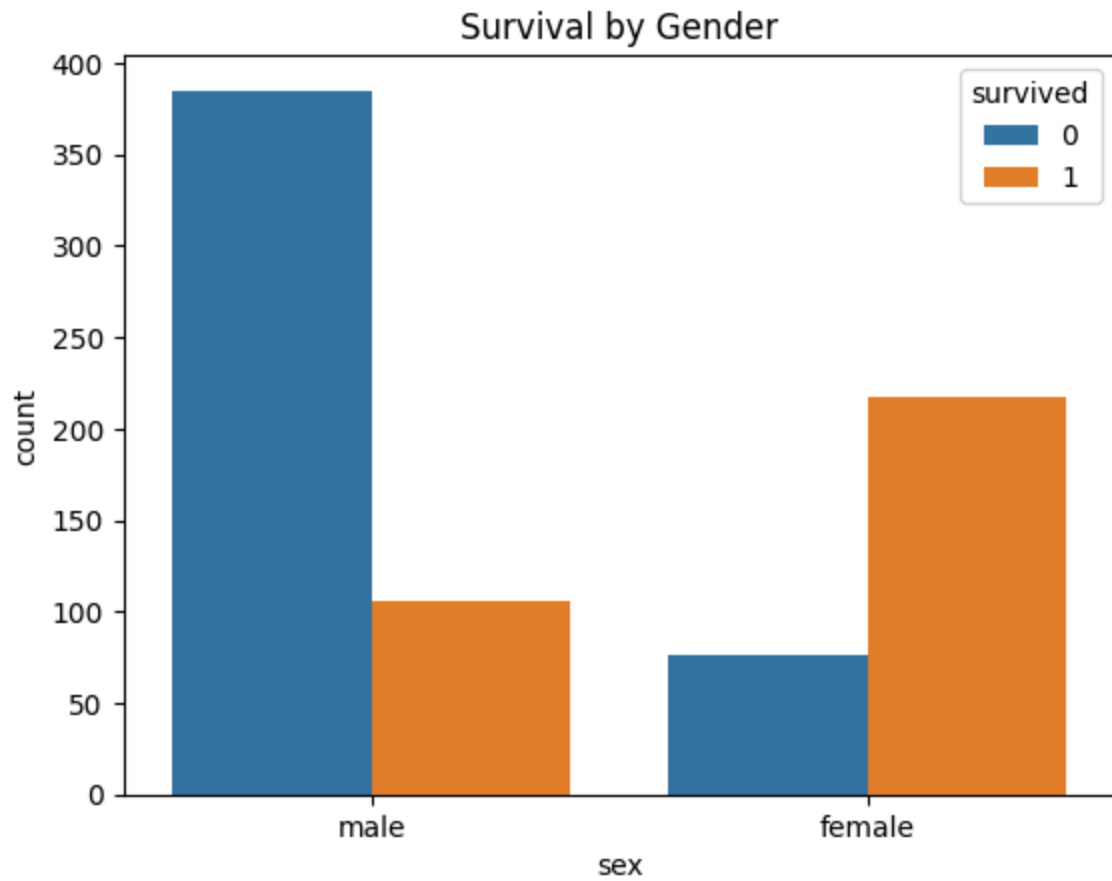


```
In [31]: #Fare distribution
sns.histplot(data=kashti, x="fare", bins=30, kde=True)
plt.title("Fare Distribution")
plt.show()
```

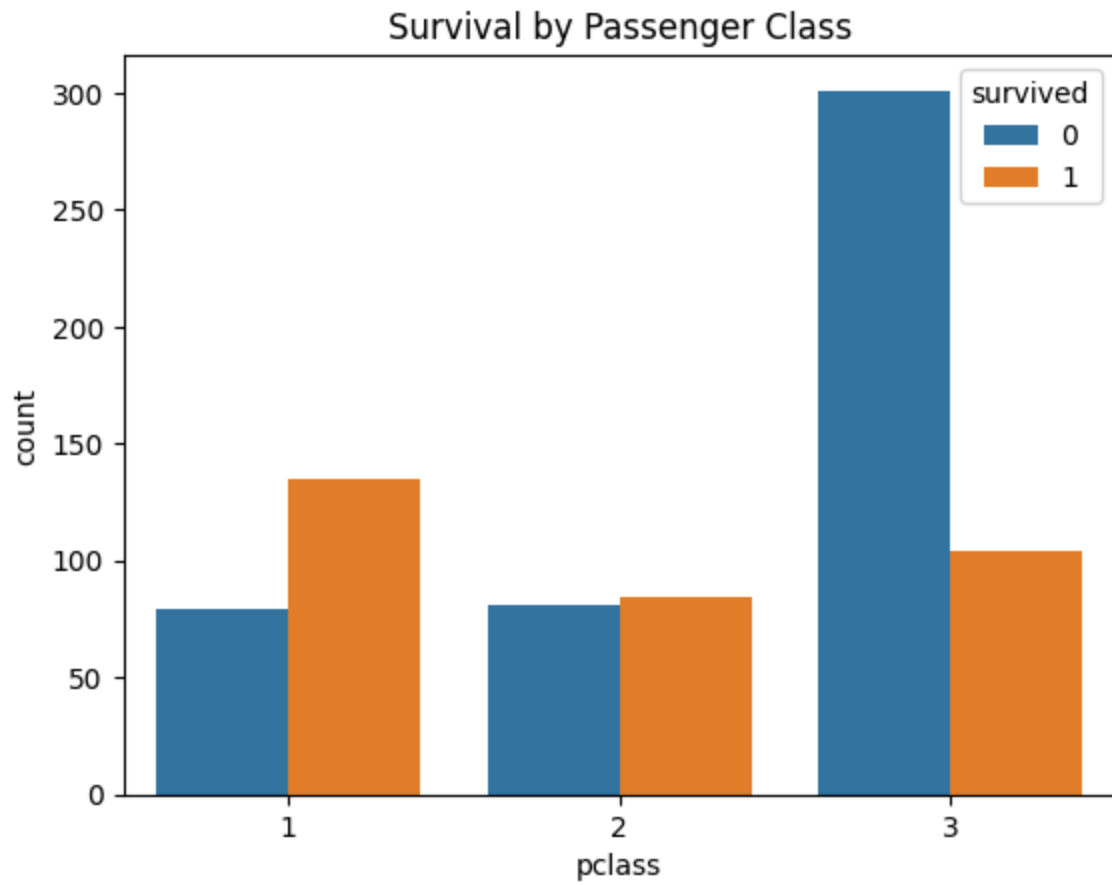


7. Bivariate Analysis

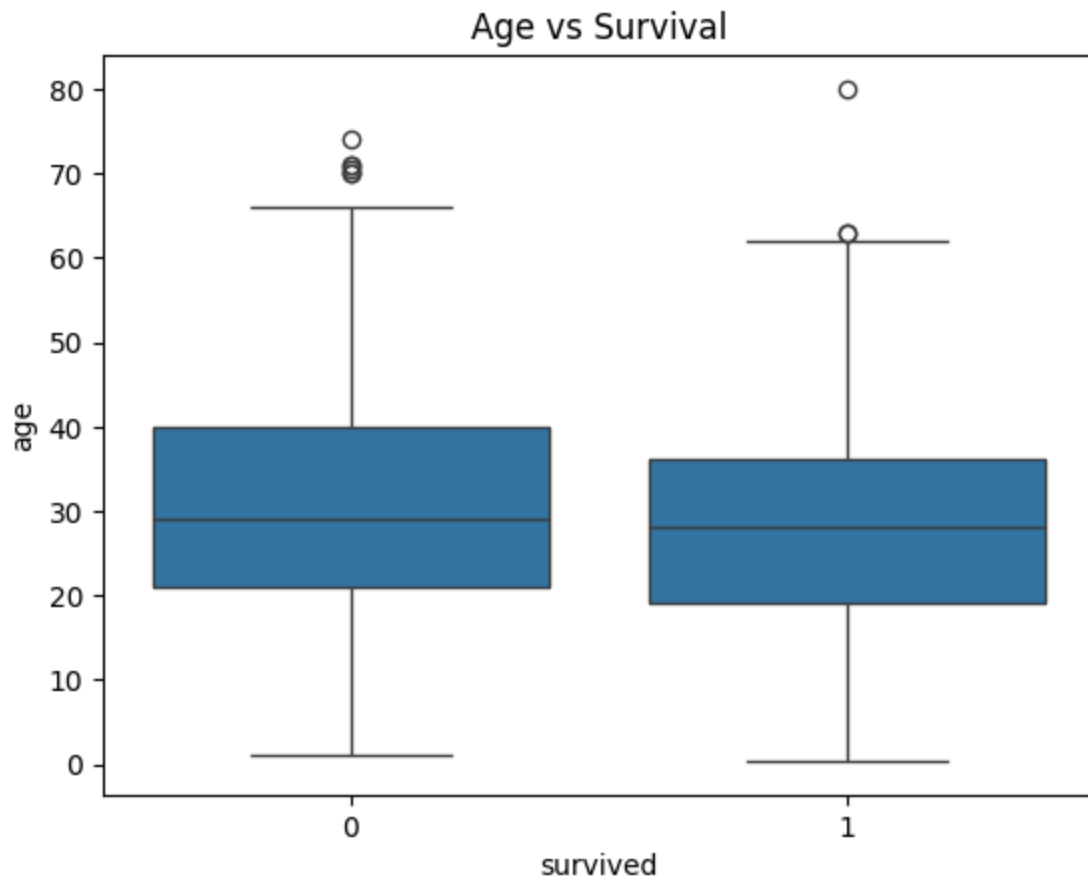
```
In [32]: #Gender vs Survival
sns.countplot(data=kashti, x="sex", hue="survived")
plt.title("Survival by Gender")
plt.show()
```



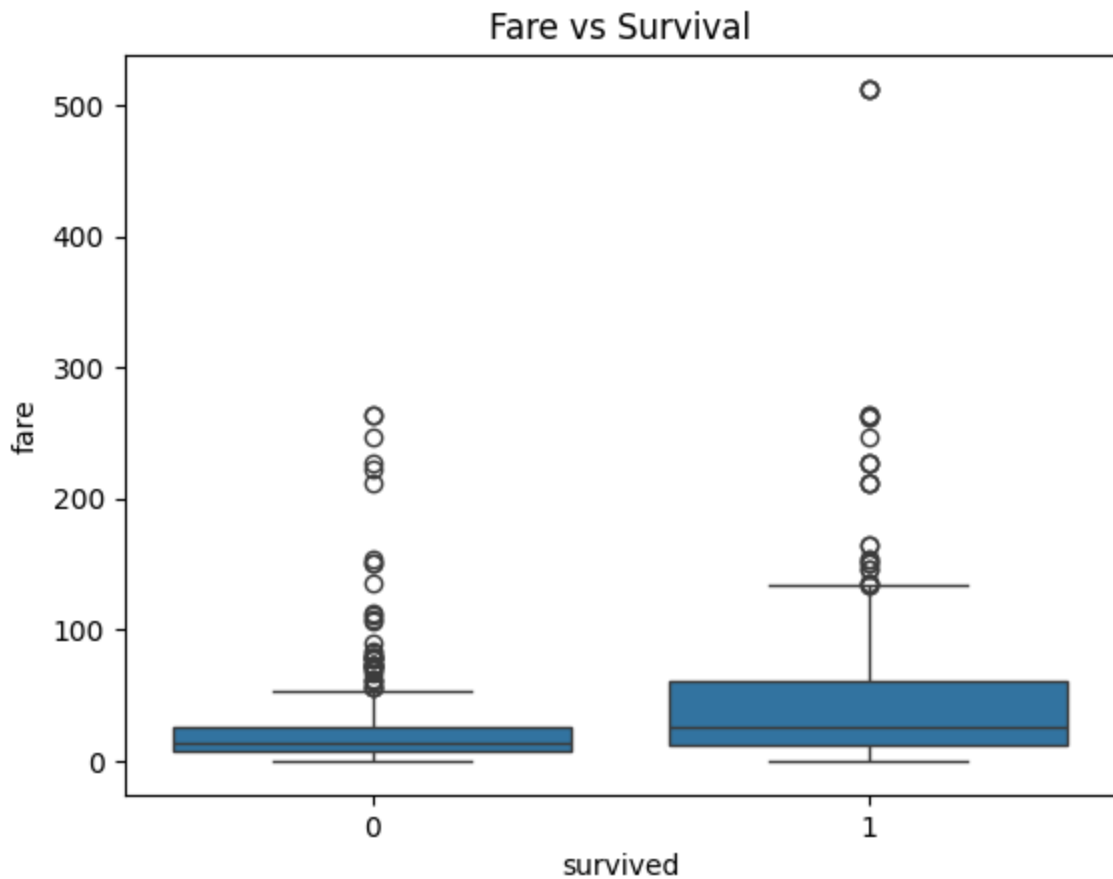
```
In [33]: # Passenger class vs Survival
sns.countplot(data=kashti, x="pclass", hue="survived")
plt.title("Survival by Passenger Class")
plt.show()
```



```
In [34]: ## Age vs Survival
sns.boxplot(data=kashti, x="survived", y="age")
plt.title("Age vs Survival")
plt.show()
```



```
In [35]: #Fare vs Survival
sns.boxplot(data=kashti, x="survived", y="fare")
plt.title("Fare vs Survival")
plt.show()
```



8. Calculate survival rates

```
In [36]: #Overall survival rate  
kashti["survived"].mean()
```

```
Out[36]: np.float64(0.4119897959183674)
```

```
In [38]: #Survival by gender  
kashti.groupby("sex")["survived"].mean()
```

```
Out[38]: sex  
female    0.740614  
male      0.215886  
Name: survived, dtype: float64
```

```
In [39]: #Survival by class  
kashti.groupby("pclass")["survived"].mean()
```

```
Out[39]: pclass  
1      0.630841  
2      0.509091  
3      0.256790  
Name: survived, dtype: float64
```

```
In [40]: #Survival by gender and class  
kashti.groupby(["sex", "pclass"])["survived"].mean()
```

```
Out[40]: sex      pclass
female 1      0.967742
        2      0.917808
        3      0.472441
male    1      0.371901
        2      0.184783
        3      0.158273
Name: survived, dtype: float64
```

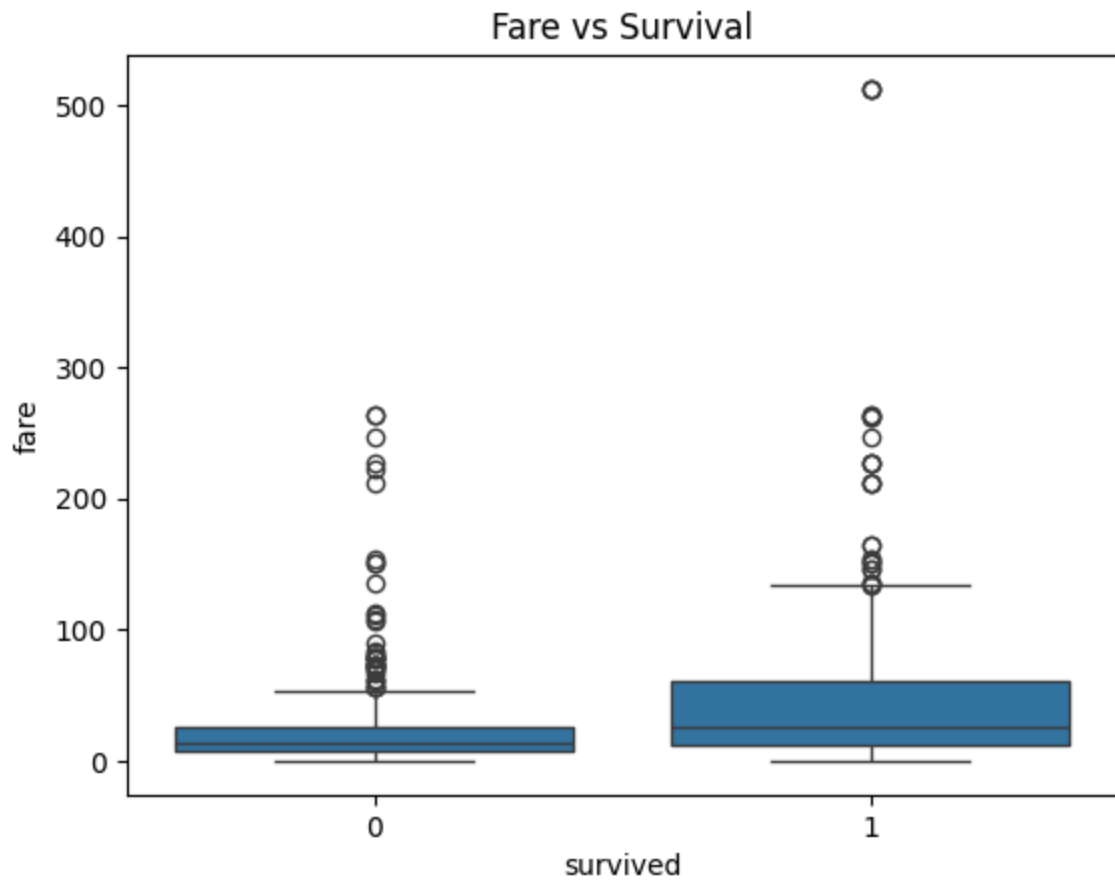
```
In [41]: survival_table = pd.pivot_table(
        kashti,
        values="survived",
        index="sex",
        columns="pclass",
        aggfunc="mean"
    )

survival_table
```

```
Out[41]:
```

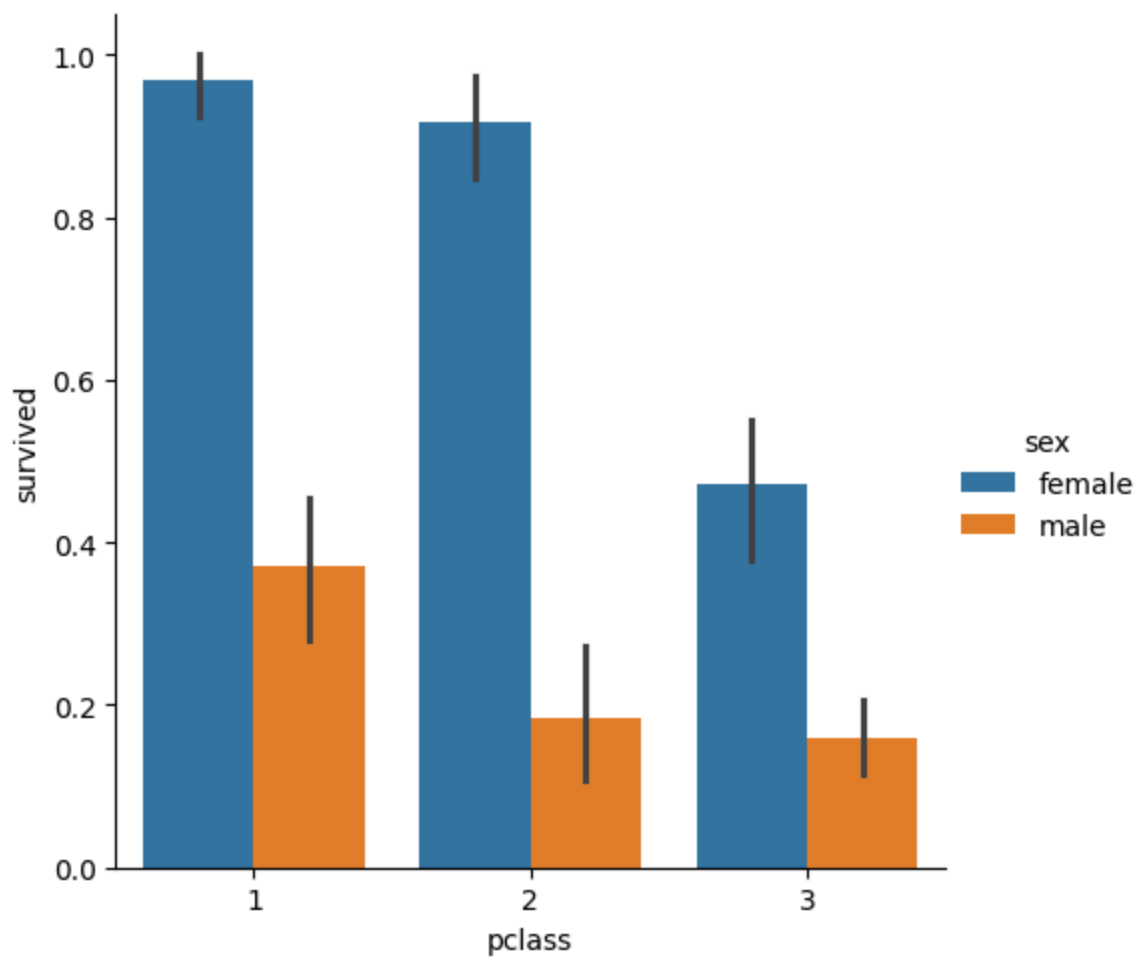
	pclass	1	2	3
sex				
female		0.967742	0.917808	0.472441
male		0.371901	0.184783	0.158273

```
In [37]: # Fare vs Survival
sns.boxplot(data=kashti, x="survived", y="fare")
plt.title("Fare vs Survival")
plt.show()
```

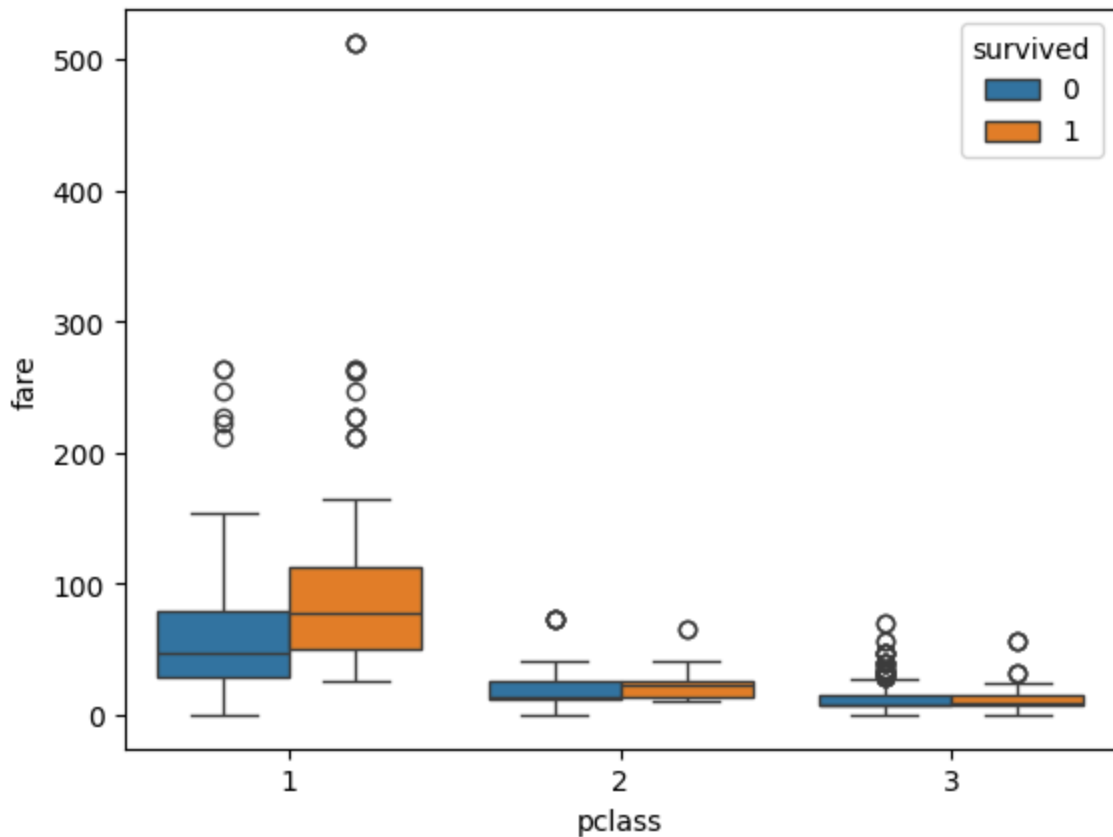



9. Multivariate Analysis

```
In [42]: sns.catplot(  
    data=kashti,  
    x="pclass",  
    y="survived",  
    hue="sex",  
    kind="bar"  
)  
  
plt.show()
```



```
In [43]: sns.boxplot(  
    data=kashti,  
    x="pclass",  
    y="fare",  
    hue="survived"  
)  
  
plt.show()
```



10. Correlation analysis

```
In [44]: numeric_kashti = kashti.select_dtypes(include=np.number)

numeric_kashti.corr()
```

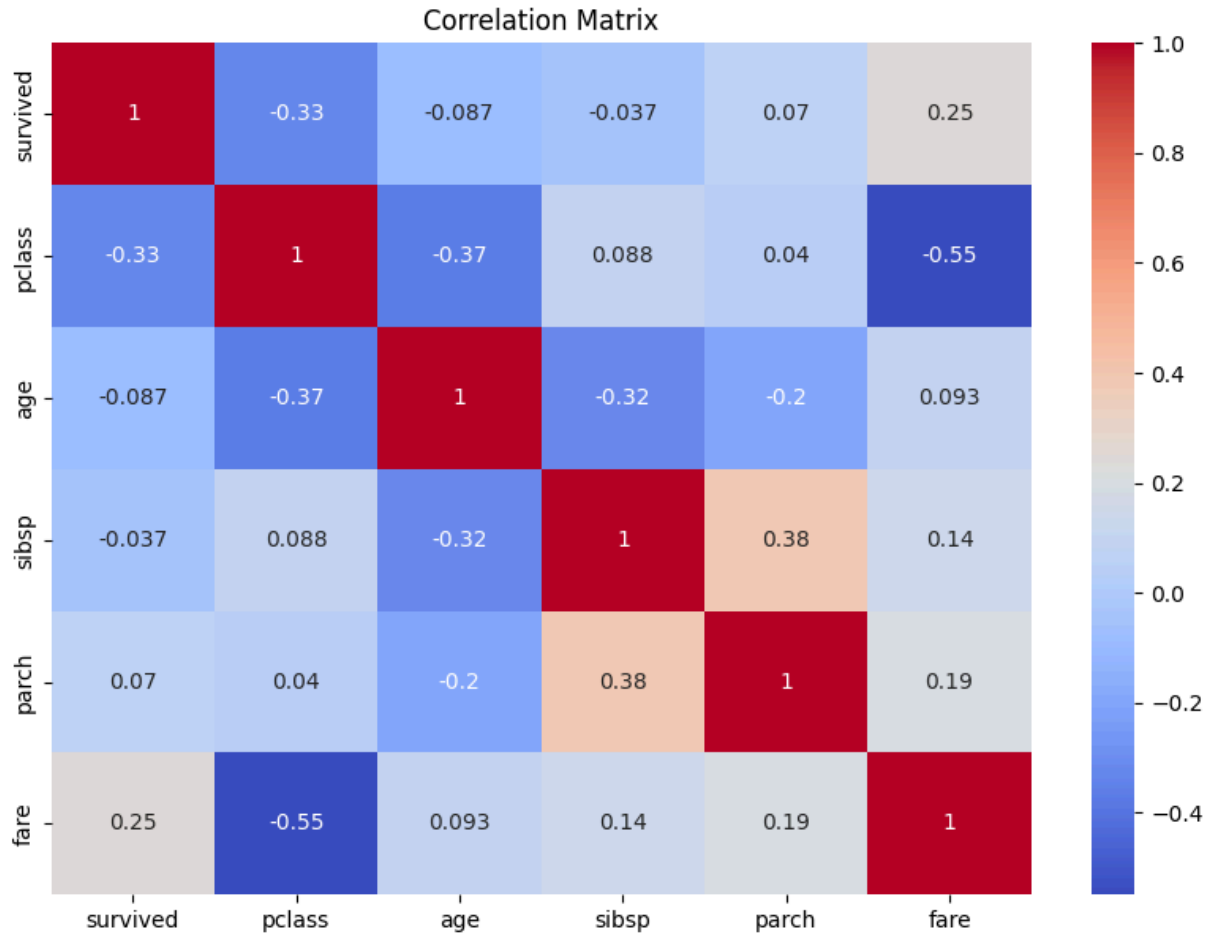
```
Out[44]:
```

	survived	pclass	age	sibsp	parch	fare
survived	1.000000	-0.332658	-0.086548	-0.036589	0.070307	0.246769
pclass	-0.332658	1.000000	-0.369361	0.088014	0.040296	-0.549216
age	-0.086548	-0.369361	1.000000	-0.315116	-0.195036	0.092707
sibsp	-0.036589	0.088014	-0.315116	1.000000	0.381433	0.135147
parch	0.070307	0.040296	-0.195036	0.381433	1.000000	0.191942
fare	0.246769	-0.549216	0.092707	0.135147	0.191942	1.000000

```
In [45]: plt.figure(figsize=(10, 7))

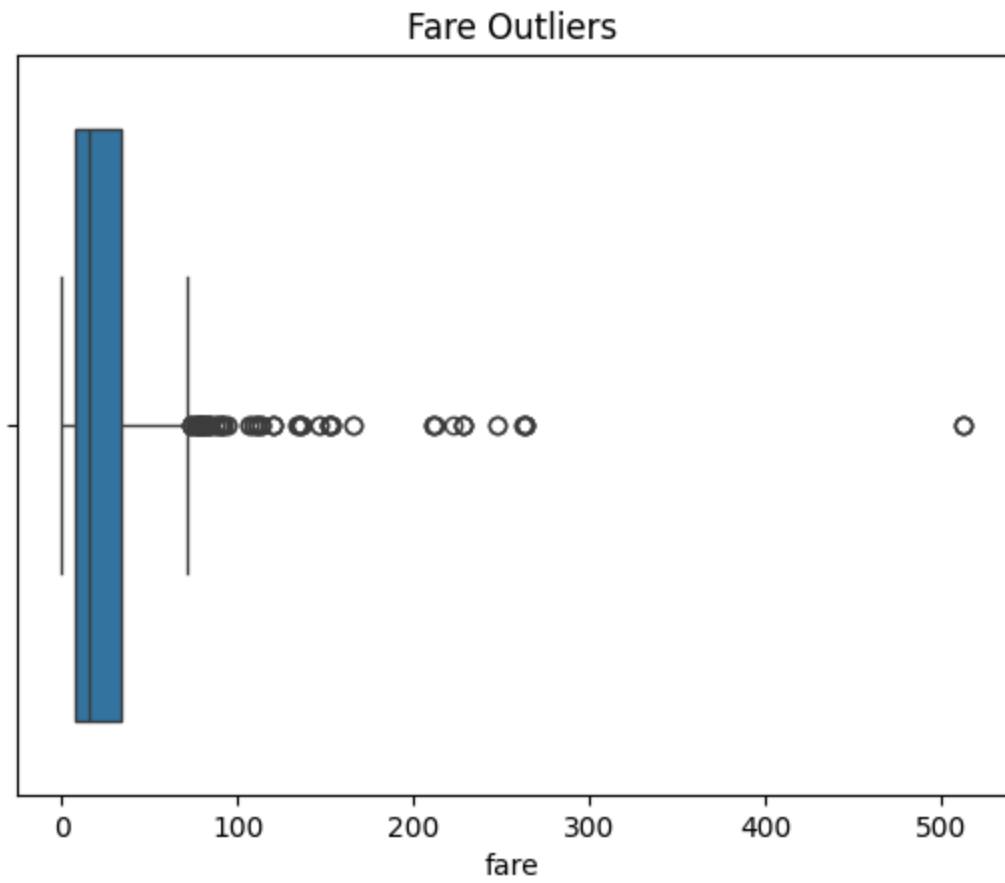
sns.heatmap(
    numeric_kashti.corr(),
    annot=True,
    cmap="coolwarm"
)
```

```
plt.title("Correlation Matrix")
plt.show()
```

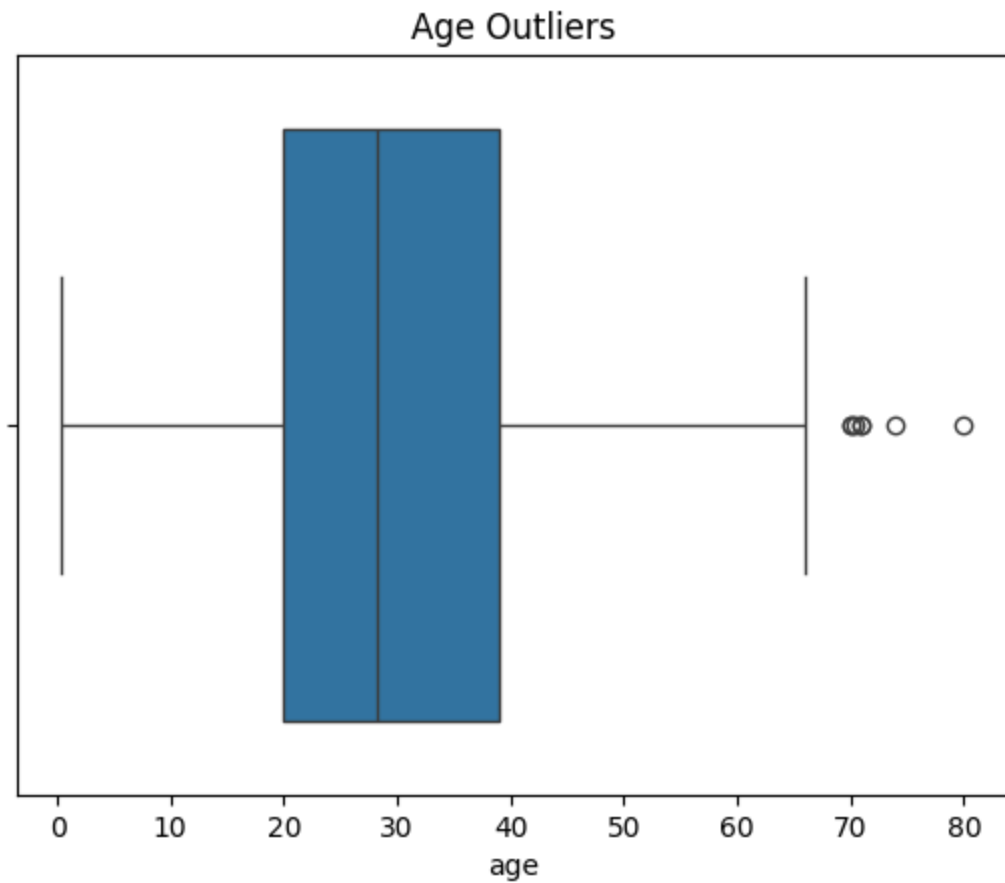


11. Outlier analysis

```
In [46]: #Fare:
sns.boxplot(data=kashti, x="fare")
plt.title("Fare Outliers")
plt.show()
```



```
In [47]: #Age:
sns.boxplot(data=kashti, x="age")
plt.title("Age Outliers")
plt.show()
```



```
In [48]: #Calculate IQR:
Q1 = kashti["fare"].quantile(0.25)
Q3 = kashti["fare"].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = kashti[
    (kashti["fare"] < lower) |
    (kashti["fare"] > upper)
]

outliers
```

```
Out[48]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adu
27	0	1	male	19.0	3	2	263.0000	S	First	man	
31	1	1	female	NaN	1	0	146.5208	C	First	woman	
34	0	1	male	28.0	1	0	82.1708	C	First	man	
52	1	1	female	49.0	1	0	76.7292	C	First	woman	
61	1	1	female	38.0	0	0	80.0000	NaN	First	woman	
...	...	...	...	...	...	...	...	...	...	...	...
829	1	1	female	62.0	0	0	80.0000	NaN	First	woman	
835	1	1	female	39.0	1	1	83.1583	C	First	woman	
849	1	1	female	NaN	1	0	89.1042	C	First	woman	
856	1	1	female	45.0	1	1	164.8667	S	First	woman	
879	1	1	female	56.0	0	1	83.1583	C	First	woman	

102 rows × 15 columns

12. Feature relationships

```
In [ ]: #Did women survive more than men?
```

```
pd.crosstab(
    kashti["sex"],
    kashti["survived"],
    normalize="index"
)
```

```
Out[ ]: survived      0      1
```

	sex	0	1
female	0.259386	0.740614	
male	0.784114	0.215886	

```
In [51]: #Did first-class passengers survive more?
```

```
pd.crosstab(
    kashti["pclass"],
    kashti["survived"],
    normalize="index"
)
```

```
Out[51]: survived      0      1
         pclass
         -----
         1  0.369159  0.630841
         2  0.490909  0.509091
         3  0.743210  0.256790
```

```
In [52]: # Did children have better survival?
kashti["Age_Group"] = pd.cut(
    kashti["age"],
    bins=[0, 12, 18, 35, 60, 100],
    labels=["Child", "Teenager", "Young Adult", "Adult", "Senior"]
)
```

```
In [53]: kashti.groupby("Age_Group", observed=True)["survived"].mean()
```

```
Out[53]: Age_Group
Child      0.573529
Teenager   0.447761
Young Adult 0.404255
Adult      0.401042
Senior     0.227273
Name: survived, dtype: float64
```

13. Feature engineering

```
In [ ]: #Create family size:
kashti["FamilySize"] = kashti["sibsp"] + kashti["parch"] + 1
```

```
In [69]: # Alone passengers
kashti["Alone"] = (kashti["FamilySize"] == 1).astype(int)
```

```
In [75]: kashti.columns
```

```
Out[75]: Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',
               'embarked', 'class', 'who', 'adult_male', 'deck', 'embark_town',
               'alive', 'alone', 'Age_Group', 'FamilySize', 'Alone'],
              dtype='object')
```

```
In [76]: kashti["survived"].value_counts()
```

```
Out[76]: survived
0      461
1      323
Name: count, dtype: int64
```

```
In [77]: kashti.groupby("sex")["survived"].mean()
```



```
Out[77]: sex
female    0.740614
male      0.215886
Name: survived, dtype: float64
```

```
In [78]: kashti.groupby("pclass")["survived"].mean()
```

```
Out[78]: pclass
1      0.630841
2      0.509091
3      0.256790
Name: survived, dtype: float64
```

```
In [79]: kashti.groupby("who")["survived"].mean()
```

```
Out[79]: who
child    0.585366
man      0.188470
woman    0.756972
Name: survived, dtype: float64
```

Add a Name column for all 891 rows

```
In [86]: def create_name(row):
         if row["age"] < 14:
             title = "Master"
         elif row["sex"] == "female":
             title = "Miss"
         else:
             title = "Mr"

         return f"{title}"
```

```
In [87]: kashti["Name"] = kashti.apply(create_name, axis=1)
```

```
In [88]: kashti[["Name", "sex", "age"]].head(10)
```

```
Out[88]:
```

	Name	sex	age
0	Mr	male	22.00
1	Miss	female	38.00
2	Miss	female	26.00
3	Miss	female	35.00
4	Mr	male	35.00
5	Mr	male	28.25
6	Mr	male	54.00
7	Master	male	2.00
8	Miss	female	27.00
9	Miss	female	14.00

```
In [89]: kashti["Title"] = kashti["Name"].str.extract(
          r",\s*([^.]+)\.",
          )[0]
```

```
In [90]: kashti[["Name", "Title"]].head(10)
```

```
Out[90]:
```

	Name	Title
0	Mr	NaN
1	Miss	NaN
2	Miss	NaN
3	Miss	NaN
4	Mr	NaN
5	Mr	NaN
6	Mr	NaN
7	Master	NaN
8	Miss	NaN
9	Miss	NaN

```
In [91]: kashti["Title"].value_counts()
```

```
Out[91]: Series([], Name: count, dtype: int64)
```

14. Missing-value treatment

```
In [62]: kashti["age"] = kashti["age"].fillna(kashti["age"].median())
```

```
In [64]: kashti["embarked"] = kashti["embarked"].fillna(
        kashti["embarked"].mode()[0]
    )
```

```
In [67]: kashti.columns
```

```
Out[67]: Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',
               'embarked', 'class', 'who', 'adult_male', 'deck', 'embark_town',
               'alive', 'alone', 'Age_Group', 'FamilySize', 'Alone'],
              dtype='object')
```

```
In [93]: kashti["CabinKnown"] = kashti["deck"].notna().astype(int)
```

```
In [95]: kashti[["deck", "CabinKnown"]].head(10)
```

```
Out[95]:
```

	deck	CabinKnown
0	NaN	0
1	C	1
2	NaN	0
3	C	1
4	NaN	0
5	NaN	0
6	E	1
7	NaN	0
8	NaN	0
9	NaN	0

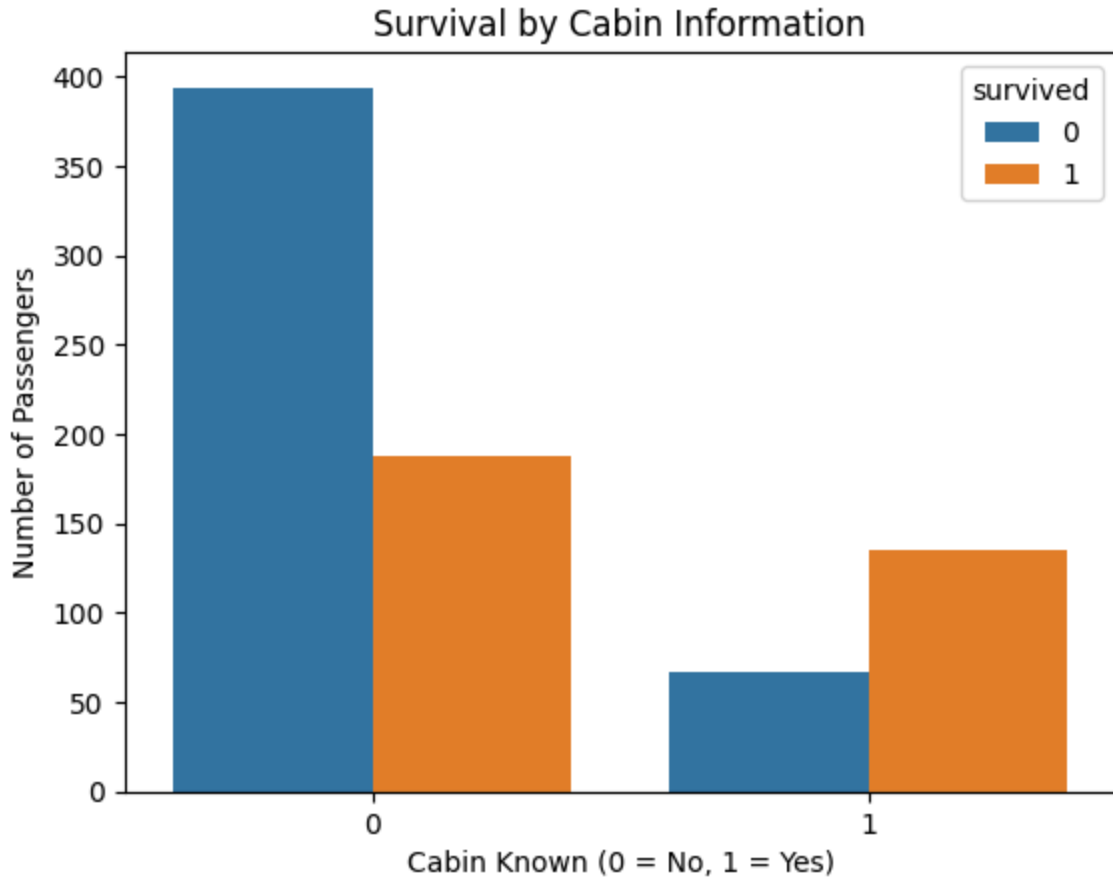
```
In [96]: kashti["CabinKnown"].value_counts()
```

```
Out[96]: CabinKnown
0      582
1      202
Name: count, dtype: int64
```

```
In [97]: sns.countplot(
        data=kashti,
        x="CabinKnown",
        hue="survived"
    )

plt.title("Survival by Cabin Information")
plt.xlabel("Cabin Known (0 = No, 1 = Yes)")
```

```
plt.ylabel("Number of Passengers")
plt.show()
```



15. Final EDA checklist

Titanic EDA | ☐ 1. Load Data | ☐ read_csv() | ☐ 2. Understand Data | ☐ head()
☐ shape | ☐ columns | ☐ info() | ☐ describe() | ☐ dtypes | ☐ 3. Data
Quality | ☐ Missing values | ☐ Duplicates | ☐ Incorrect data types | ☐ Outliers
☐ 4. Univariate Analysis | ☐ Age | ☐ Fare | ☐ Sex | ☐ Pclass | ☐
Survived | ☐ 5. Bivariate Analysis | ☐ Sex vs Survived | ☐ Pclass vs Survived | ☐
Age vs Survived | ☐ Fare vs Survived | ☐ 6. Multivariate Analysis | ☐ Sex + Pclass
+ Survived | ☐ Age + Sex + Survived | ☐ Fare + Pclass + Survived | ☐ 7. Feature
Engineering | ☐ FamilySize | ☐ Alone | ☐ Age_Group | ☐ Title | ☐ 8.
Prepare for Machine Learning | ☐ Encode categorical variables | ☐ Select features | ☐
Train/test split | ☐ Build model

The key EDA questions for Titanic

Think of EDA as asking the data questions:

"Kashti mein kon log zinda bachay?"

1. Did women survive more than men?
2. Did children survive more than adults?
3. Did 1st-class passengers survive more than 2nd/3rd class?
4. Did fare relate to survival?
5. Did traveling alone vs with family affect survival?
6. Did age + gender + class together explain survival better?
7. Which variables have the strongest relationship with Survived?